

Python 單元測試與突變測試 自動生成系統

改進實作報告

專案：vscode_llm_unit_test

報告日期：2026-08-31

本報告彙整已完成的程式改進、真實驗證結果與後續部署建議。核心方向是：維持領域中立，以技能卡按需特化，並讓生成測試在進入專案前先通過可執行與突變有效性的檢查。

版本範圍：截至 cdd1ef8 文件用途：專題報告與後續實作交接

1. 執行摘要

本專案目標是讓不同能力等級的雲端模型與本地模型，都能參與 Python 單元測試和突變測試的自動化流程。同時，系統不再依賴任何特定業務詞彙或單一函式模板，而是由靜態 AST、動態執行結果、模型能力資格與技能卡共同決定最安全且最有效的測試策略。

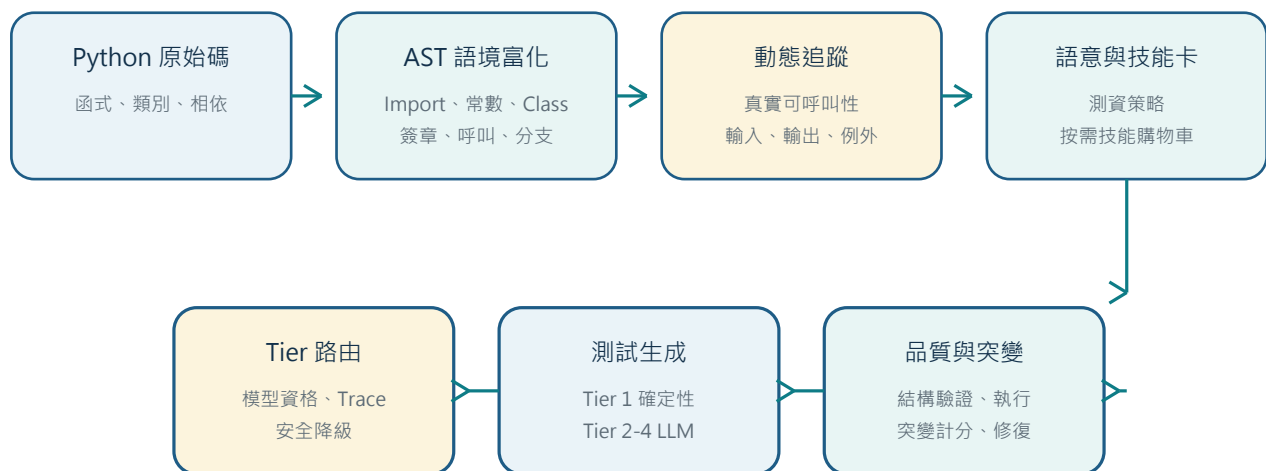
已完成的主軸：AST 語境富化、按需技能卡、可驗證的動態追蹤、Tier 安全路由、生成測試品質閘門、內建突變後備引擎與雲端金鑰安全管理。

設計原則

原則	落地方式
領域中立	核心程式與共用 Prompt 不含 bmi、token、jwt 等業務關鍵字；以純度測試防止回歸污染。
按需特化	語意分析師依程式特徵挑選技能卡，例如切片、浮點、非同步、檔案 Mock 與時間凍結。
語境完整	把檔案 Import、常數、類別初始化、函式簽章與相依呼叫一併交給生成流程。
先驗證再採用	生成結果需通過結構檢查、實際 unittest 預檢與突變成效評估，失敗時才交由 Reviewer 定點修復。

2. 已完成的系統架構

系統將「理解程式」、「取得真實行為」、「選擇能力」、「生成測試」與「驗證測試」拆開處理。這降低了小模型或格式不穩定模型直接決定錯誤斷言的風險。



流程說明：AST

先提供完整靜態語境；動態追蹤再提供可重現的輸入、輸出與例外。語意分析師挑選技能卡與測資策略，Tier 路由器依模型資格與 Trace

可用性選擇生成方式。最後由驗證器、實際執行與突變測試共同決定測試是否可接受。

3. 已實作的關鍵改進

面向	已完成內容	帶來的效果
技能卡與 Prompt	建立切片、浮點精度、字典或元組回傳、例外、async coroutine、mock_open、datetime freezing 等技能卡；語意失敗時仍可由語法特徵保底推導。	避免針對單一業務函式寫死規則，同時提供足夠的測試專業知識。
AST 語境	擷取 Import 與 alias、被引用模組常數、類別 bases 與 attributes、__init__ 參數和 self 賦值、Name 和 Attribute 呼叫、完整函式簽章。	LLM 可知道 self.db、型別、常數與可 Patch 的相依，而非在語境孤島中猜測。
相對匯入	支援 from .helpers、from ..shared、from . import sibling，並以目標檔案位置解析相依。	套件式 Python 專案的相依分析更準確。
動態追蹤	結合 caller 字面量、型別註記、條件式分支導向輸入；保留 keyword-only 參數；等待 async 結果；避免不可信的 object.__new__ 類別追蹤。	產生的是已驗證的真實 I/O 與例外案例，不是模型憑空猜出的斷言。
Tier 路由	模型資格與 provider + model 綁定。資格失敗時，手動要求 Tier 2-4 也會安全降到 Tier 1；無 Trace 時停止並說明原因。	低能力模型不能繞過品質保護，系統仍可輸出可靠的確定性測試。
生成驗證	要求 unittest 結構、test 方法、斷言、實際呼叫目標；允許 async 測試；拒絕重定義或用 sys.modules 偽造目標模組。	防止看似通過、實際沒有測到原始碼的假測試。
突變測試	整合 mutatest 或 mutmut；Windows 或不相容環境自動改用不改寫原始碼的 AST 後備引擎。	即使外部工具不可用，也能評估邊界與條件測試是否真的有效。
雲端與安全	Cloud Gemini UI 分成 API key 名稱、API model、API key；儲存後清空輸入；金鑰使用 SecretStorage，不寫入設定或 Git。	方便多金鑰與多模型管理，並降低機密外洩風險。

4. 不同模型能力的 Tier 使用策略

Tier 並不是單純依模型名稱切分，而是由小型自我描述測試確認該 provider + model 是否能穩定輸出可解析、可執行的 unittest JSON。這使本地 Ollama 與 Cloud Gemini 使用相同的品質入口。

Tier	適用條件	生成模式	安全措施
Tier 1	模型未通過資格，或使用小型本地模型	完全依已驗證 Trace 產生確定性 unittest，正確序列化位置參數與關鍵字參數，支援類別、static 與 class method。	沒有 Trace 時明確停止，不假裝產生有效測試。
Tier 2	通過資格的小型或中型 Instruct 模型	AST 語境 + 技能卡 + 基礎語意策略生成。	結構、目標呼叫與實際 unittest 預檢。
Tier 3	通過資格的中型模型，且有外部相依	生成 scaffold 並以真正使用點進行 Mock 或 Patch。	禁止只 Patch 定義處而未影響被測程式。
Tier 4	通過資格的強模型或雲端模型	完整語境、依賴鏈、Reviewer 自癒與存活突變定點修補。	仍受同一組驗證器與突變計分約束。

重要行為：使用者即使手動選擇較高 Tier，未通過資格的模型也不會直接產生不可信的 LLM 測試。系統會降至 Tier 1；若沒有已驗證的追蹤資料，則安全停止並提示先完成追蹤。

5. 突變測試的可靠性設計

突變類型	後備引擎做法	目的
比較運算	替換 <、<=、>、>=、==、!= 的比較方向或關係。	檢查邊界值與條件分支是否被覆蓋。
算術與數值	替換常見算術運算，並改變數值常數。	檢查計算公式與常數假設是否有斷言保護。
布林邏輯	替換 and、or、not 與布林常數。	檢查多條件邏輯與否定分支。
作用範圍	只對選取的函式或類別方法建立暫存副本突變。	避免改動使用者原始碼，也避免把無關模組計入分數。

若環境可使用 mutatest 或 mutmut，系統優先採用其確定性計分；若 Windows Python 版本不相容，內建後備引擎仍會提供可用的突變訊號。Reviewer 的工作是修補「存活突變」，而非盲目重寫整份測試。

6. 已取得的真實驗證證據

檢查項目	結果	意義
專案自動測試	TypeScript 49 項通過；Python 15 項通過。	主要流程與新保護邏輯已有回歸測試。
編譯與靜態檢查	compile 通過；lint 0 errors，保留 49 個既有 warnings。	目前改進未引入編譯或 lint error。
動態追蹤	complex_test 的 service_auth.login_user 與 service_order.checkout_order 已成功取得真實回傳及 ValueError 案例。	分支輸入、成功結果與例外可作為 Tier 1 或 LLM 的可靠依據。
本地模型探針	實測 codegemma:2b 回傳 {"code": "unittest"}，未符合結構化 unittest 資格。	模型被正確標示為未合格，不能跳過 Tier 路由保護。
生成測試防護	測試會檢查 assert、目標呼叫、Python AST 語法與真正 unittest 執行。	避免空斷言、假呼叫或替換目標模組的測試混入成果。

結論：系統已可在模型能力不足時維持可執行的 Tier 1 產出；要完整驗證 Tier 2-4 的生成品質，仍需以一個已通過資格的 Cloud 模型或更強的本地 Instruct 模型，跑一次端到端實測。

7. Git 變更紀錄摘要

已依專案規則，在每個完成且通過測試的功能段落建立本機 Git commit，訊息同時包含英文與中文；未進行遠端推送。以下為代表性紀錄。

Commit	內容
9a6af74	加入語意測資策略與技能卡。
fd56707	驗證生成測試並富化 AST 語境。
561b828	驗證模型可生成 unittest 的資格。
b7c54e4	驗證生成測試確實呼叫目標。

Commit	內容
23fc679	將後備突變限定於選定目標。
4448180	隔離 provider + model 的生成資格。
cdd1ef8	解析 Python 相對匯入相依。

8. 尚待完成與建議的下一步

優先級	建議工作	完成標準
P0	以新的 Cloud Gemini key 搭配 ListModels 找到支援 generateContent 的實際模型，或指定更強的本地 Instruct 模型，完成模型資格探針。	資格通過，並完成一次 Tier 2 或 Tier 4 端到端生成、執行、突變修補。
P1	在 CI 或相容 Python 環境安裝 mutatest 或 mutmut，與後備引擎結果交叉比較。	外部計分與後備計分都有報告，差異可追蹤。
P1	補上 property、setter、decorator 與更深層套件相對匯入的專案化測試案例。	AST、Trace 與生成器對這些 Python 特性都有回歸測試。
P2	依真實失敗 log 擴充技能卡，而不是在核心 Prompt 加入特定領域規則。	每張新卡都有觸發條件、範例與無污染的測試。

9. 部署與操作建議

本地開發時，Cloud Gemini 金鑰應只輸入 VS Code 擴充功能的安全儲存欄位；不要放入 .env、settings.json、測試結果或 Git commit。每次更換 Cloud 模型後先執行連線及模型資格檢查，再選擇 Tier。Ollama 本地模型也應先跑資格探針；小型模型未合格並不代表不能使用，而是會被安排到可驗證 Trace 的 Tier 1。

產出流程建議：選取目標函式 - 先取得 AST 與動態 Trace - 確認模型資格 - 生成測試 - 執行預檢 - 進行突變測試 - 對存活突變做 Reviewer 修補。每個已驗證的實作段落都應更新中文變更日誌並建立雙語 Git commit。

本次報告所描述的是已實作與已驗證的能力；尚未宣稱所有模型皆已完成 Tier 2-4 的真實端到端驗證。此差距已保留為下一階段的明確驗收項目。